

文脈等価性の局所解析手法 とその展開

室屋 晃子

(京都大学 数理解析研究所)

文脈等価性の局所解析手法 とその展開

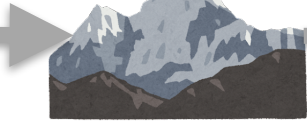
(駆け出し研究者、作問を学ぶ)

室屋 晃子

(京都大学 数理解析研究所)



成果：局所解析法



課題：限定的で「汚い」

紆余曲折

自己紹介

- 2012.4—2014.3 東京大学理学部情報科学科
- 2014.4—2016.3 東京大学大学院情報理工学研究科
コンピュータ科学専攻 修士課程
- 2016.4—2020.7  バーミンガム大学
コンピュータ科学専攻 博士課程
- 2018.4— 京都大学数理解析研究所 助教

研究分野

- 理論計算機科学
 - プログラム意味論
 - 操作的意味論：実行過程に注目する
 - 表示の意味論：振る舞いを数学的に特徴づける
 - 公理の意味論：満たされる性質に注目する

文脈等価性

- 操作的意味論での基本的・重要な問題
- プログラム片の等しさの定式化
 - 「プログラムの一部を変更しても振る舞いが変わらないか？」
- 示せると嬉しい
 - リファクタリング・コンパイラ最適化の正しさの保証

```
let x = 1 + 2 in  
let y = 3 in  
x + x;
```

```
let x = 3 in  
let y = 3 in  
x + x;
```

```
let x = 3 in  
x + x;
```

文脈等価性

- 操作的意味論での基本的・重要な問題
- プログラム片の等しさの定式化
 - 「プログラムの一部を変更しても振る舞いが変わらないか？」
- 示せると嬉しいが、難しい
 - 項 t と u が文脈等価である
 - $\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$
 - 文脈 C を愚直に調べ切るのはほぼ不可能

文脈等価性

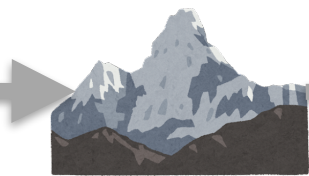
- 操作的意味論での基本的・重要な問題
- プログラム片の等しさの定式化
 - 「プログラムの一部を変更しても振る舞いが変わらないか？」
- 示せると嬉しいが、難しい
 - 項 t と u が文脈等価である
 - $\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$
- 文脈 C の機能に注目した特徴づけをするアプローチ
 - applicative bisimilarity [Abramsky '90]
 - logical relations [Plotkin '73] [Statman '85]

文脈等価性

- 操作的意味論での基本的・重要な問題
- プログラム片の等しさの定式化
 - 「プログラムの一部を変更しても振る舞いが変わらないか？」
- 示せると嬉しいが、難しい
 - 項 t と u が文脈等価である
 - $\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$
- 文脈 C の必要な部分を局所的に解析するアプローチ
 - 局所解析手法 [M. '20]



成果：局所解析法



課題：限定的で「汚い」

紆余曲折

局所解析手法の流れ

- 文脈等価性を証明する

- 項 t と u が文脈等価である

$\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$

1. 操作的意味論を決め、振る舞いを定義する

2. 項 t, u と任意の文脈 C との相互作用を解析する

局所解析手法の流れ

- 文脈等価性を証明する

- 項 t と u が文脈等価である

$\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$

1. 操作的意味論を決め、振る舞いを定義する

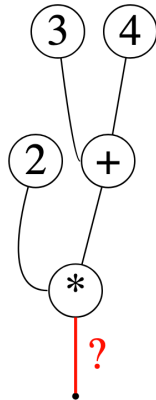
→ トークン式グラフ書き換え系を使う

2. 項 t, u と任意の文脈 C との相互作用を解析する

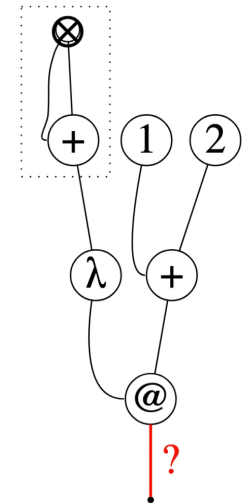
操作的意味論：トークン式グラフ書き換え系

- プログラム → グラフ表現
 - 抽象構文木 + スコープ情報 - 変数名

$2 \times (3 + 4)$



$(\lambda x. x + x) (1 + 2)$



操作的意味論：トークン式グラフ書き換え系

- プログラム評価 → グラフ表現上のトークン操作の繰り返し
 - 3種のトークン操作で評価の流れを表す
 - (?) 評価箇所を**探索**する
 - (✓) 探索を**終了**しバックトラックする
 - (ㄥ) **計算**する → グラフ表現の書き換え

操作的意味論：トークン式グラフ書き換え系

- プログラム評価 → グラフ表現上のトークン操作の繰り返し
 - 3種のトークン操作で評価の流れを表す
 - (?) 探索
 - (✓) 終了
 - (↱) 計算

操作的意味論：トークン式グラフ書き換え系

- プログラム評価 → グラフ表現上のトークン操作の繰り返し
 - 3種のトークン操作で評価の流れを表す
 - (?) 探索
 - (✓) 終了
 - (↱) 計算

トークンが移動

操作的意味論：トークン式グラフ書き換え系

- プログラム評価 → グラフ表現上のトークン操作の繰り返し
- 3種のトークン操作で評価の流れを表す

- (?) 探索

トークンが移動

- (✓) 終了

- (↱) 計算

トークン周辺のグラフ表現の書き換え

操作的意味論：トークン式グラフ書き換え系

- プログラム評価 → グラフ表現上のトークン操作の繰り返し

- 3種のトークン操作で評価の流れを表す

- (?) 探索

トークンが移動

- (✓) 終了

- (↴) 計算

トークン周辺のグラフ表現の書き換え

- 例： $2 \times (3 + 4)$

- 例： $(\lambda x. x + x) (1 + 2)$

- <https://tnttodda.github.io/Spartan-Visualiser/>

操作的意味論：トークン式グラフ書き換え系

- (?) 探索

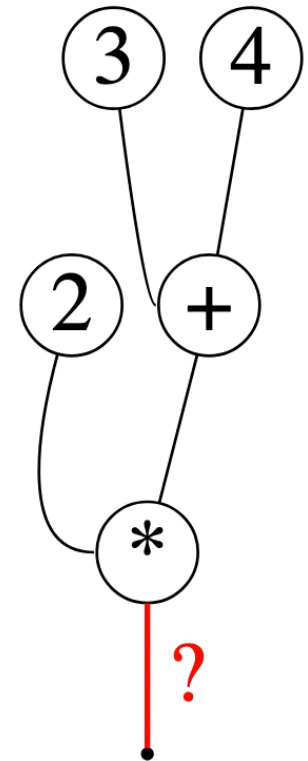
トークンが移動

- (✓) 終了

- (↱) 計算

トークン周辺のグラフ表現の書き換え

- 例： $2 \times (3 + 4)$



- <https://tnttodda.github.io/Spartan-Visualiser/>

操作的意味論：トークン式グラフ書き換え系

- (?) 探索

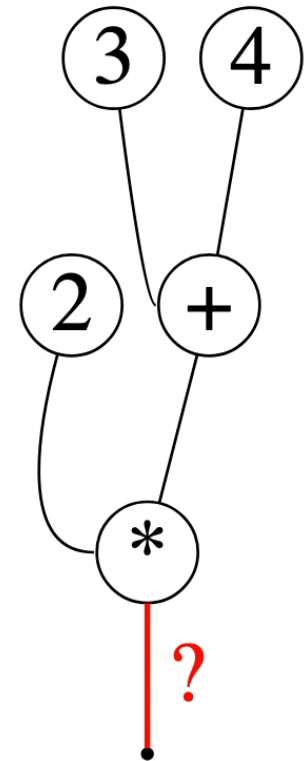
トークンが移動

- (✓) 終了

- (↱) 計算

トークン周辺のグラフ表現の書き換え

- 例： $2 \times (3 + 4)$



- <https://tnttodda.github.io/Spartan-Visualiser/>

操作的意味論：トークン式グラフ書き換え系

- (?) 探索

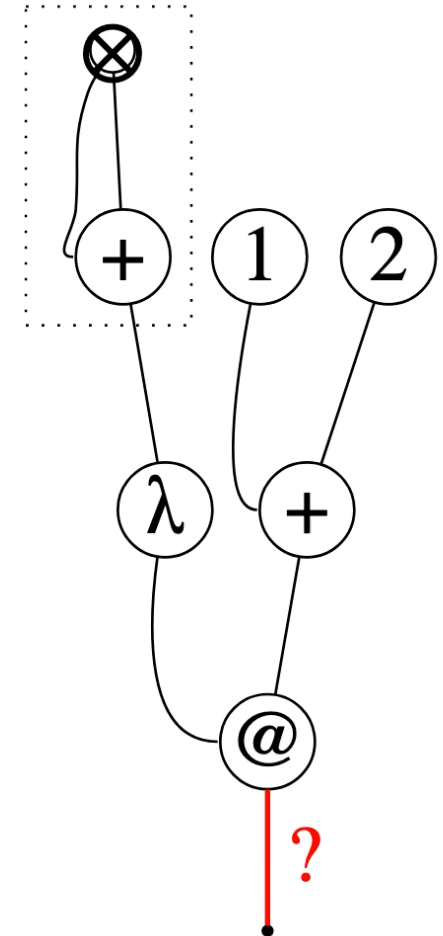
トークンが移動

- (✓) 終了

- (ㄥ) 計算

トークン周辺のグラフ表現の書き換え

- 例： $(\lambda x. x + x) (1 + 2)$



- <https://tnttodda.github.io/Spartan-Visualiser/>

操作的意味論：トークン式グラフ書き換え系

- (?) 探索

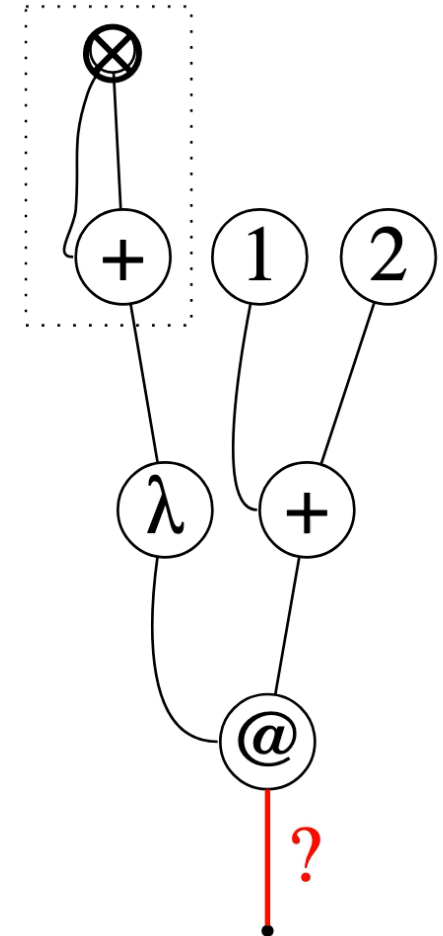
トークンが移動

- (✓) 終了

- (㏞) 計算

トークン周辺のグラフ表現の書き換え

- 例： $(\lambda x. x + x) (1 + 2)$



- <https://tnttodda.github.io/Spartan-Visualiser/>

局所解析手法の流れ

- 文脈等価性を証明する
 - 項 t と u が文脈等価である

$\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$

1. 操作的意味論を決め、振る舞いを定義する

→ トークン式グラフ書き換え系の正常終了を観測する

2. 項 t, u と任意の文脈 C との相互作用を解析する

局所解析手法の流れ

- 文脈等価性を証明する

- 項 t と u が文脈等価である

$\iff \forall \text{文脈 } C. C[t] \text{ と } C[u] \text{ は振る舞いが等しい}$

1. 操作的意味論を決め、振る舞いを定義する

→ トークン式グラフ書き換え系の正常終了を観測する

2. 項 t, u と任意の文脈 C との相互作用を解析する

→ グラフ G, H とトークン操作との相互作用を解析する

移動 or 周辺の書き換え

文脈等価性の証明：相互作用の解析

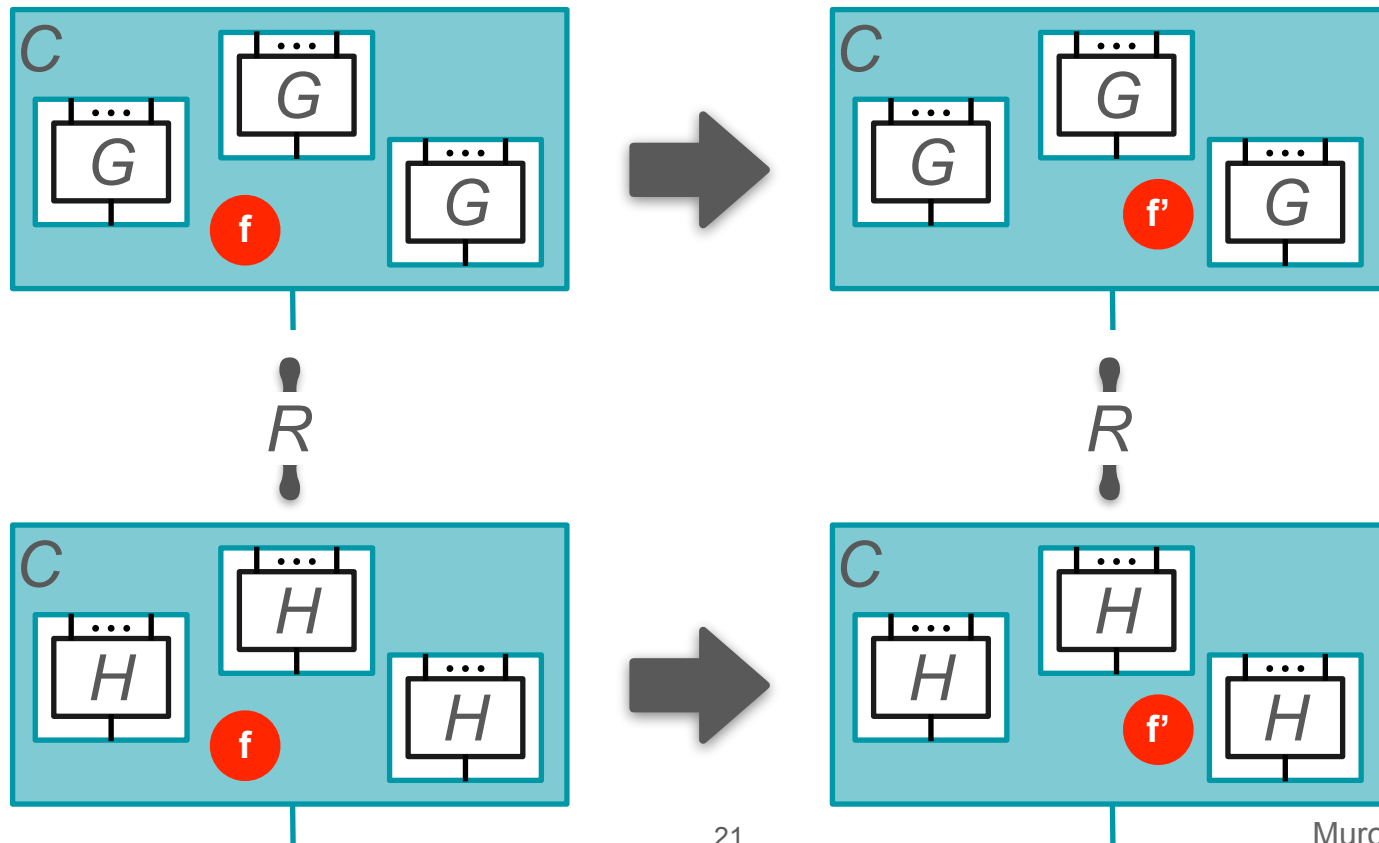
- グラフ G, H とトークン操作（移動or周辺の書き換え）との局所的な相互作用
- 相互作用の3類型
 - A. 単純な相互作用
 - B. グラフ G, H に依る相互作用
 - C. 非自明な相互作用

文脈等価性の証明：相互作用の解析

- グラフ G, H とトークン操作（移動or周辺を書き換え）との局所的な相互作用

A. 単純な相互作用：トークンが文脈 C 内を移動するだけ

- G と H の違いに依らず同じ相互作用が可能

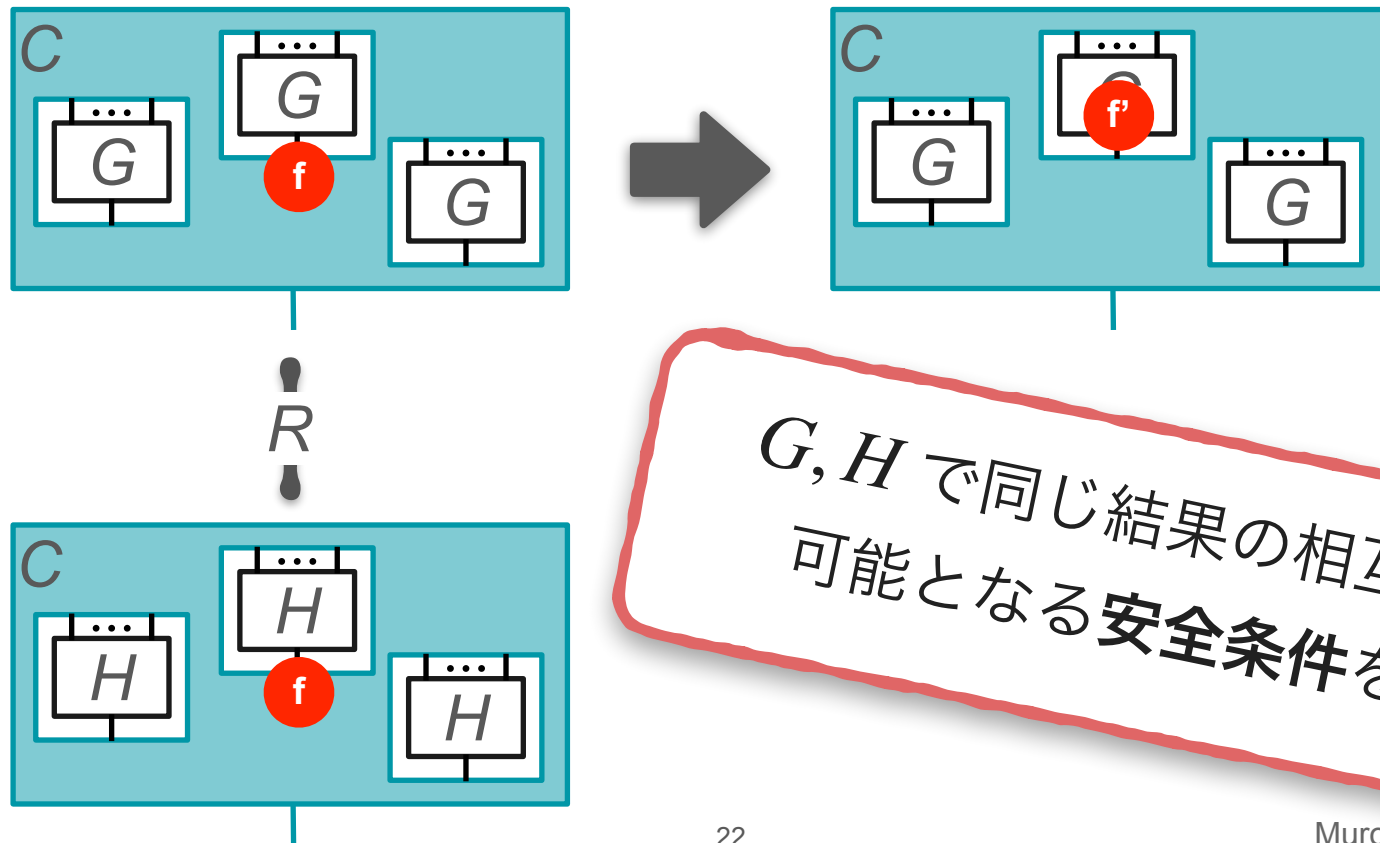


文脈等価性の証明：相互作用の解析

- グラフ G, H とトークン操作（移動or周辺の書き換え）との局所的な相互作用

B. グラフ G, H に依る相互作用：トークンが G, H 内へ移動する

- G と H の違いが相互作用の違いに直結する

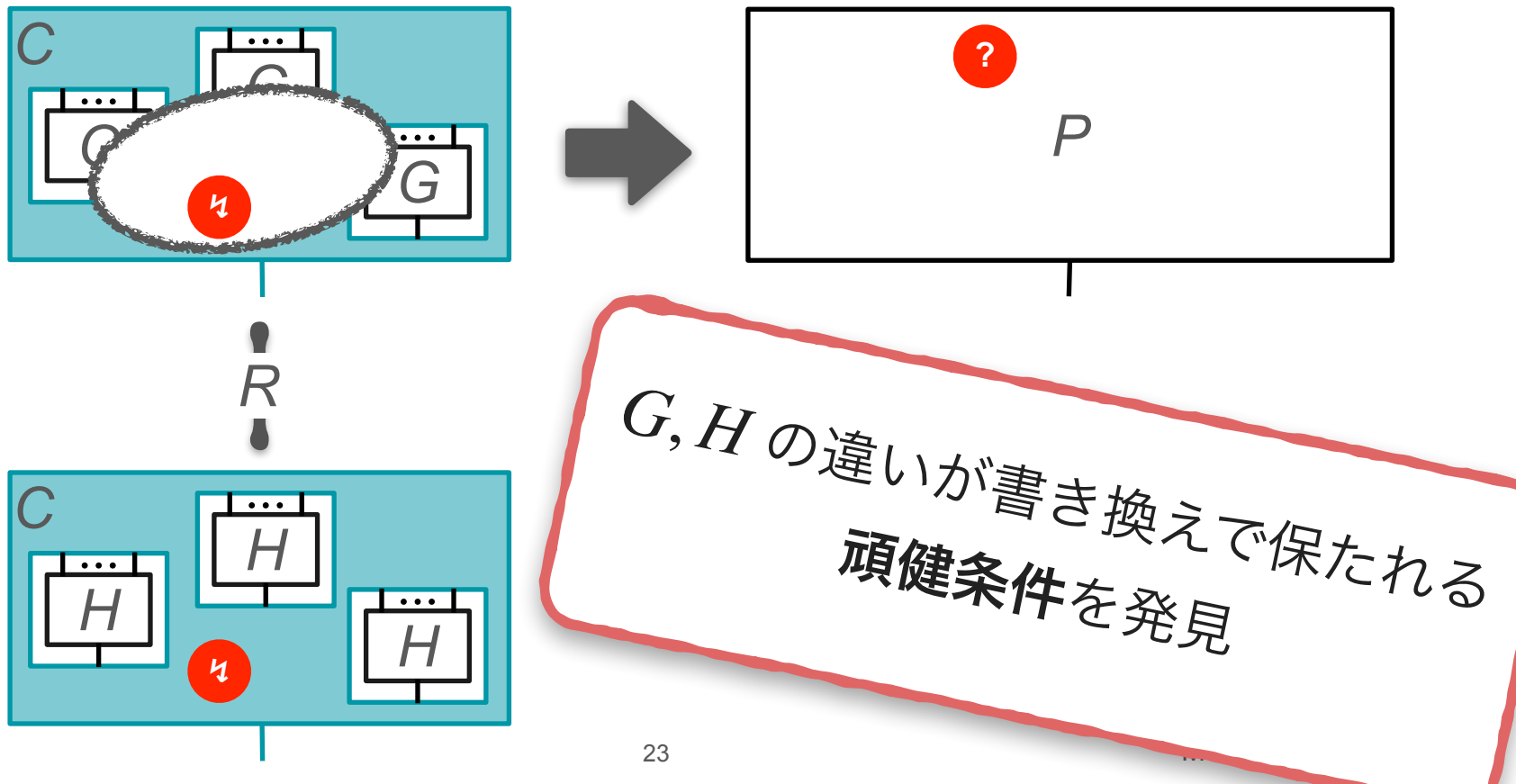


文脈等価性の証明：相互作用の解析

- グラフ G, H とトークン操作（移動or周辺を書き換え）との局所的な相互作用

C. 非自明な相互作用：トークン周辺での書き換えが起こる

- G と H の違いが相互作用の違いに関わりうる



相互作用の局所的解析による文脈等価性の証明

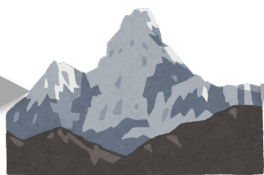
- グラフ G, H とトークン操作（移動or周辺の書き換え）との局所的な相互作用を解析
- 相互作用の3類型
 - A. 単純な相互作用： G, H の違いに依らず同じ相互作用
 - B. グラフ G, H に依る相互作用： G, H が**安全**ならば同じ結果の相互作用
 - C. 非自明な相互作用： G, H が**頑健**ならば同じ相互作用
- **定理** G, H が安全かつ頑健ならば、 G と H は文脈等価である
 - ただし、決定的なプログラムに対してのみ有効

局所解析手法のまとめ

- グラフ G, H とトークン操作（移動or周辺の書き換え）との局所的な相互作用を解析することで文脈等価性を証明する
- **定理** G, H が安全かつ頑健ならば、 G と H は文脈等価である
 - ただし、決定的なプログラムに対してのみ有効



成果：局所解析法



課題：限定的で「汚い」



紆余曲折

局所解析手法の大きな課題

- グラフ G, H とトークン操作（移動or周辺の書き換え）との局所的な相互作用を解析することで文脈等価性を証明する
- 定理 G, H が安全かつ頑健ならば、 G と H は文脈等価である
 - ただし、決定的なプログラムに対してのみ有効
- **課題 (1)** 適用範囲が限定的
 - 可 : 参照、継続
 - 不可 : I/O、非決定的選択、確率的選択

局所解析手法の大きな課題

- グラフ G, H とトークン操作（移動or周辺の書き換え）との局所的な相互作用を解析することで文脈等価性を証明する
- 定理 G, H が安全かつ頑健ならば、 G と H は文脈等価である
 - ただし、決定的なプログラムに対してのみ有効
- **課題 (2)** 手法そのものが「汚い」
 - 本質が見えにくい
 - トークン式グラフ書き換え系は本当に必要か？
 - 拡張性に欠ける
 - I/O、非決定的選択、確率的選択はどう扱えばよいか？

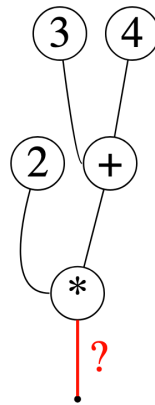
課題解決のアイデア

- アイデア (1) 由緒正しい項書き換え系を使う
- アイデア (2) より抽象的なオートマトンを使う

課題解決のアイデア (1) 項書き換え系を使う

- より理論的に成熟した操作的意味論を使いたい
- 解法? : トークン式グラフ書き換え系を項書き換え系で表す
 - LMNtal [Ueda '09] が使えそう
 - プログラム → グラフ表現 → LMNtal項

$2 \times (3 + 4)$

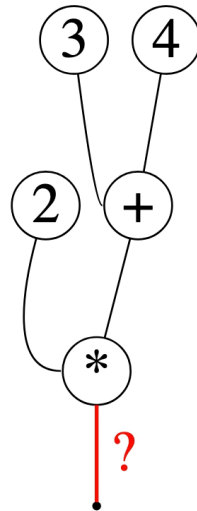


```
V2 = 2,  
V3 = 3,  
V4 = 4,  
Plus = +(V3, V4),  
Times = x(V2, Plus),  
Root = ?(Times).
```

課題解決のアイデア (1) 項書き換え系を使う

- より理論的に成熟した操作的意味論を使いたい
- 解法? : トークン式グラフ書き換え系を項書き換え系で表す
 - LMNtal [Ueda '09] が使えそう
 - プログラム評価 → トークン操作 → LMNtal項の書き換え

```
2 × (3 + 4)
-> 2 × 7
-> 14
```



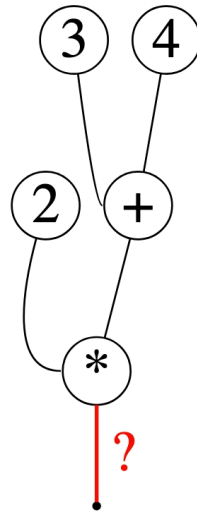
```
X = ?(Plus), Plus = +(Y,Y')
:- X = +(Z,Y'), Z = ?(Y).
```

```
X = ♯(Plus), Plus = +(A,B)
:- X = ?(Z), Z = A+B
```

課題解決のアイデア (1) 項書き換え系を使う

- より理論的に成熟した操作的意味論を使いたい
- 解法? : トークン式グラフ書き換え系を項書き換え系で表す
 - LMNtal [Ueda '09] が使えそう
 - プログラム評価 → トークン操作 → LMNtal項の書き換え

```
2 × (3 + 4)
-> 2 × 7
-> 14
```



```
X = ?(Plus), Plus = +(Y,Y')
:- X = +(Z,Y'), Z = ?(Y).
```

```
X = ♯(Plus), Plus = +(A,B)
:- X = ?(Z), Z = A+B
```

課題解決のアイデア (1) 項書き換え系を使う

- より理論的に成熟した操作的意味論を使いたい
- 解法？：トークン式グラフ書き換え系を項書き換え系で表す
 - LMNtal [Ueda '09] が使えそう
 - 文脈等価性の局所解析手法による証明 → ?
 - 部分グラフとトークン操作との局所的な相互作用の解析 → ?

課題解決のアイデア (1) 項書き換え系を使う

- より理論的に成熟した操作的意味論を使いたい
- 解法? : トークン式グラフ書き換え系を項書き換え系で表す
 - LMNtal [Ueda '09] が使えそう
 - 文脈等価性の局所解析手法による証明 → ?
 - 部分グラフとトークン操作との局所的な相互作用の解析 → ?
- **問題点** : 項書き換えは構造合同の下で行われる

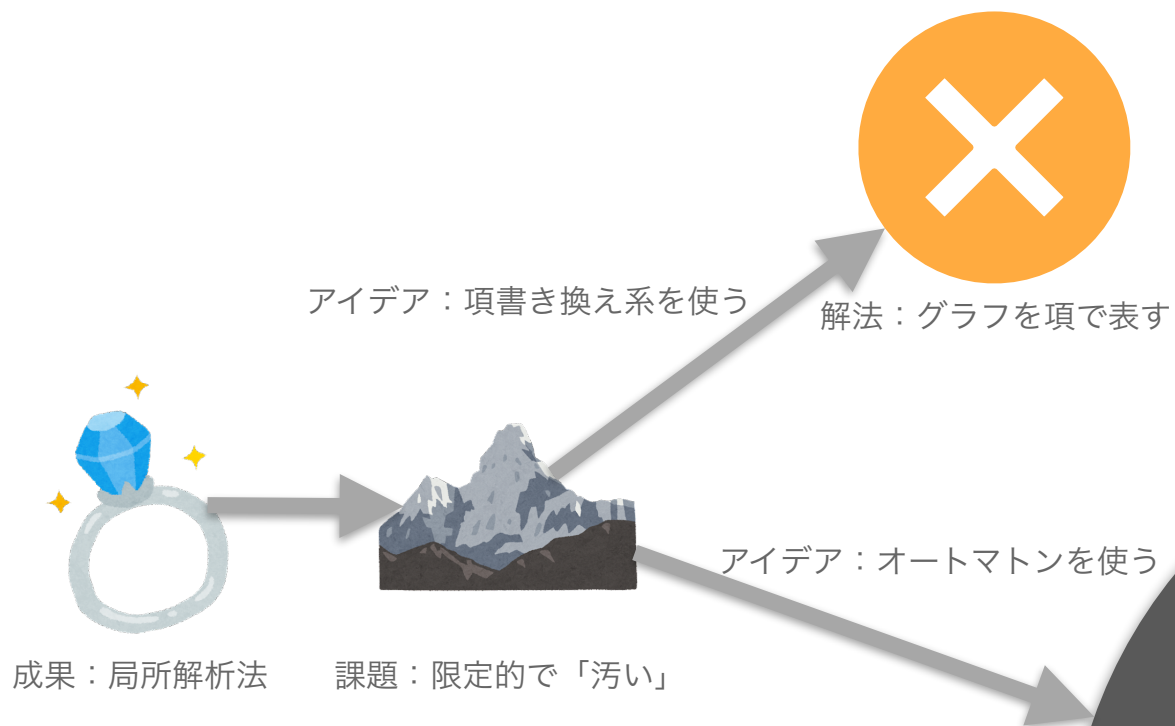
K. Ueda / Theoretical Computer Science 410 (2009) 4784–4800

(E1) $\mathbf{0}, P \equiv P$	(E2) $P, Q \equiv Q, P$	(E3) $P, (Q, R) \equiv (P, Q), R$
(E4) $P \equiv P[Y/X]$ if X is a local link of P		
(E5) $\frac{P \equiv P'}{P, Q \equiv P', Q}$	(E6) $\frac{P \equiv P'}{\{P\} \equiv \{P'\}}$	
(E7) $X = X \equiv \mathbf{0}$	(E8) $X = Y \equiv Y = X$	
(E9) $X = Y, P \equiv P[Y/X]$ if P is an atom and X occurs free in P		
(E10) $\{X = Y, P\} \equiv X = Y, \{P\}$ if exactly one of X and Y occurs free in P		

Fig. 4. Structural congruence on LMNtal processes.

課題解決のアイデア (1) 項書き換え系を使う

- より理論的に成熟した操作的意味論を使いたい
- **解法?**：トークン式グラフ書き換え系を項書き換え系で表す
 - LMNtal [Ueda '09] が使えそう
 - 文脈等価性の局所解析手法による証明 → **?**
 - 部分グラフとトークン操作との局所的な相互作用の解析 → **?**
- **問題点**：項書き換えは構造合同の下で行われる
 - 項の局所的な解析が難しい



紆余曲折

課題解決のアイデア (2) オートマトンを使う

- 局所解析手法の肝は「Q-数え模倣関係」
- **定理** G, H が安全かつ頑健ならば、
 G と H は文脈等価である
- ただし、決定的なプログラムに対してのみ有効

課題解決のアイデア (2) オートマトンを使う

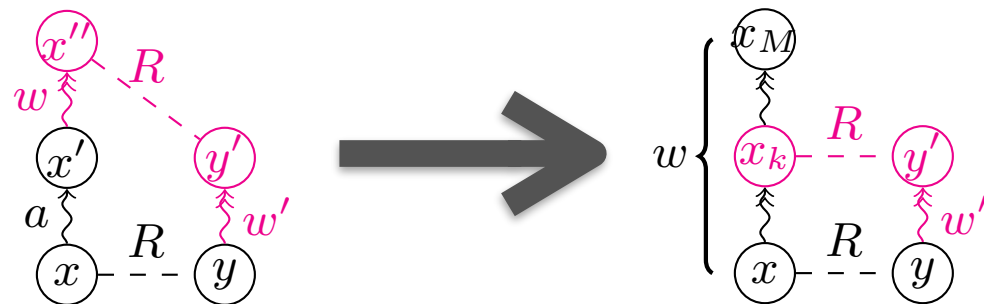
- 局所解析手法の肝は「Q-数え模倣関係」
- **定理** G, H が安全かつ頑健ならば、
 G, H 間にQ-数え模倣関係が存在し
 G と H は文脈等価である
- ただし、決定的なプログラムに対してのみ有効

課題解決のアイデア (2) オートマトンを使う

- 局所解析手法の肝は「Q-数え模倣関係」
- 定理 G, H 間にQ-数え模倣関係が存在したら、
 G と H は文脈等価である
- ただし、分岐のないオートマトンに対してのみ有効

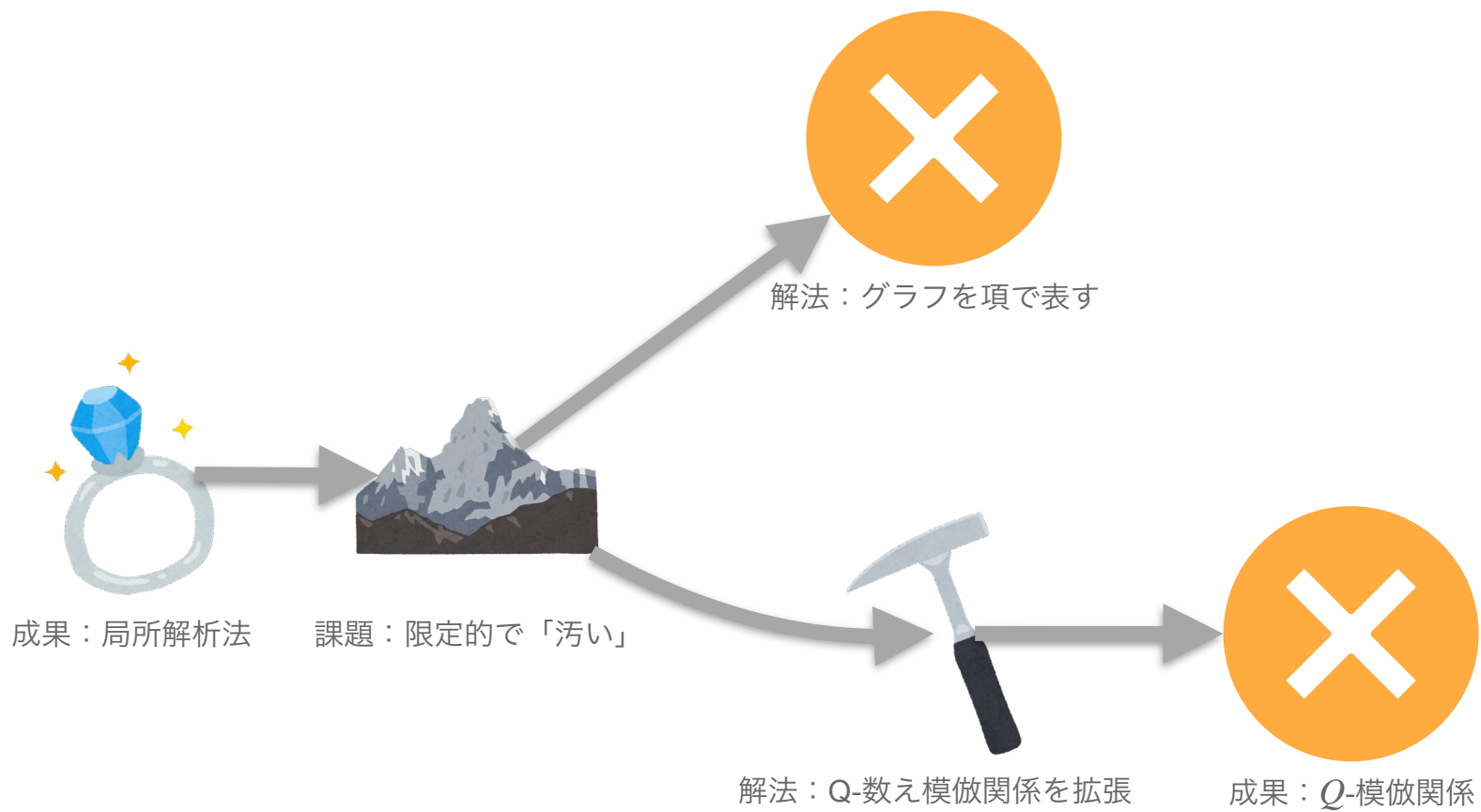
課題解決のアイデア (2) オートマトンを使う

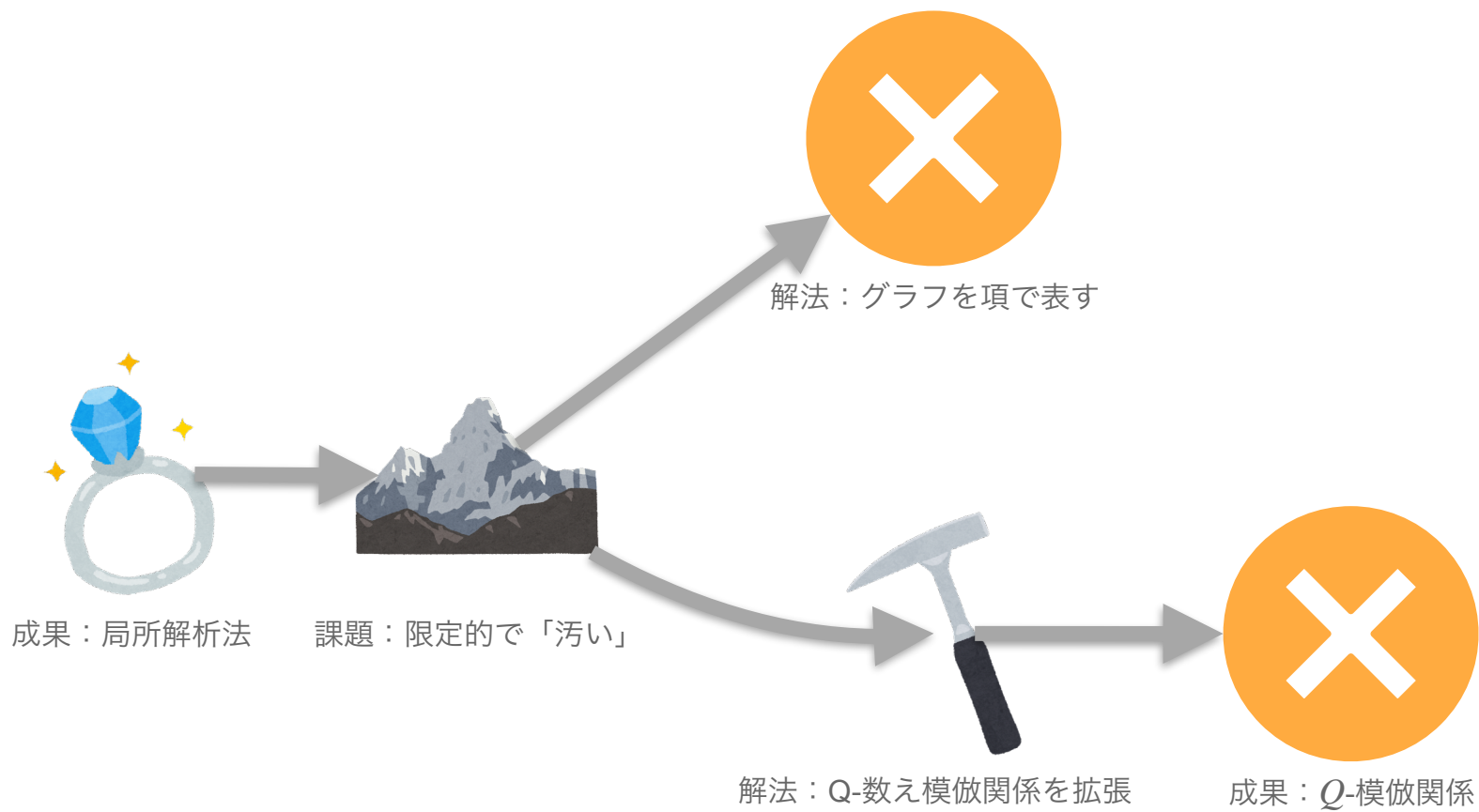
- 局所解析手法の肝は「Q-数え模倣関係」
- **定理** G, H 間にQ-数え模倣関係が存在したら、
 G と H は文脈等価である
- ただし、分岐のないオートマトンに対してのみ有効
- 観察：Q-数え模倣関係は分岐のあるオートマトンにも定義はできる
- 解法：分岐のあるオートマトンでも使えるようにQ-数え模倣関係を拡張する



課題解決のアイデア (2) オートマトンを使う

- 局所解析手法の肝は「Q-数え模倣関係」
- **定理** G, H 間にQ-数え模倣関係が存在したら、
 G と H は文脈等価である
 - ただし、分岐のないオートマトンに対してのみ有効
- 観察：Q-数え模倣関係は分岐のあるオートマトンにも定義はできる
- 解法：分岐のあるオートマトンでも使えるようにQ-数え模倣関係を拡張する
- 成果：「 Q -模倣関係」
 - CALCO '21 Early Ideasで途中経過を発表
 - FoSSaCS '22に論文投稿 → Reject 「技術的内容はあるが筋書きが不明」





打開のヒント

「その話は結局何をやっているの？」

「その話」

- **課題**：局所解析手法は限定的で「汚い」
- 局所解析手法の肝は「Q-数え模倣関係」
- **定理** G, H 間にQ-数え模倣関係が存在したら、
 G と H は文脈等価である
 - ただし、分岐のないオートマトンに対してのみ有効
- **観察**：Q-数え模倣関係は分岐のあるオートマトンにも定義はできる
- **解法**：分岐のあるオートマトンでも使えるようにQ-数え模倣関係を拡張する

「その話」

- 課題：局所解析手法は限定的で「汚い」
- 局所解析手法の肝は「Q-数え模倣関係」
- 定理 G, H 間にQ-数え模倣関係が存在したら、
 G と H は文脈等価である
 - ただし、分岐のないオートマトンに対してのみ有効
- 観察：Q-数え模倣関係は分岐のあるオートマトンにも定義はできる
- 解法：分岐のあるオートマトンでも使えるようにQ-数え模倣関係を拡張する

千葉先生

「要するに何をやっているの？」

「その話」

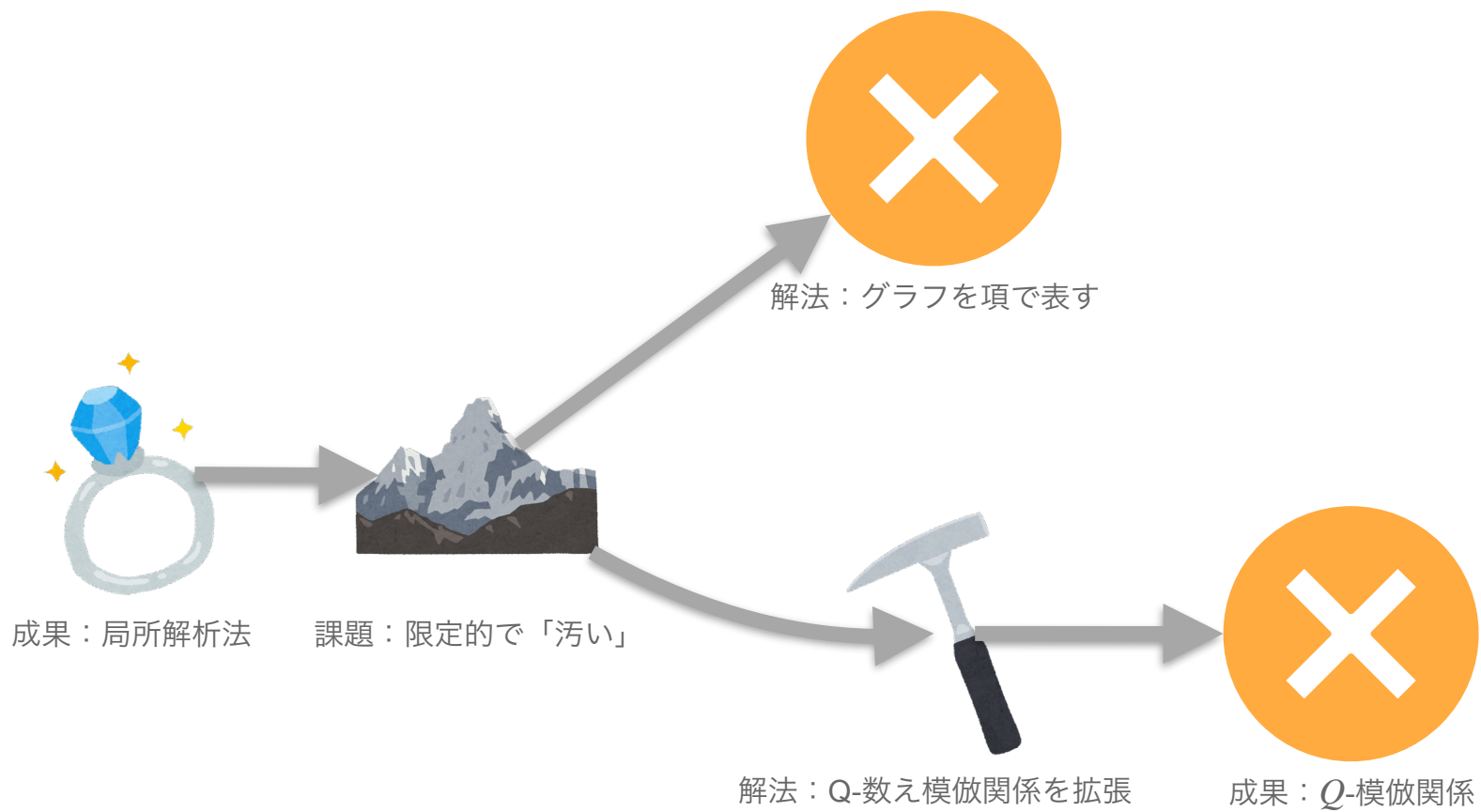
- 課題：局所解析手法は限定的で「汚い」
- 局所解析手法の肝は「Q-数え模倣関係」
- **定理** G, H 間にQ-数え模倣関係が存在したら、
 G と H は文脈等価である
 - ただし、分岐のないオートマトンに対してのみ有効
- 観察：Q-数え模倣関係は分岐のあるオートマトンにも定義はできる
- 解法：分岐のあるオートマトンでも使えるようにQ-数え模倣関係を拡張する

千葉先生

「要は文脈等価性を模倣関係で
特徴づけたいんだよね？」

筋書きの改善

- 課題：局所解析手法は限定的で「汚い」
- 問題：文脈等価性を特徴づける模倣関係を作る
- 解法：分岐のあるオートマトンでも使えるようにQ-数え模倣関係を拡張する
- 成果：「 Q -模倣関係」
 - 可：I/O、非決定的選択
 - 不可：確率的選択
 - 論文執筆中





学び：
問題を適切に設定できれば
筋書きが見えてくる

項書き換え系再び

- **課題**：局所解析手法は限定的で「汚い」
- **解法**：~~トークン式グラフ書き換え系を項書き換え系で表す~~

項書き換え系再び

- 課題：局所解析手法は限定的で「汚い」
- 問題：局所解析手法を項書き換え系で再現する
- 解法：~~トークン式グラフ書き換え系を項書き換え系で表す~~

項書き換え系再び

- 課題：局所解析手法は限定的で「汚い」
- 問題：局所解析手法を項書き換え系で再現する
- 解法：~~トークン式グラフ書き換え系を項書き換え系で表す~~
- 解法：プログラムを項とみなして項書き換え系を使う

新たな課題の発見

- 課題：局所解析手法は限定的で「汚い」
- 問題：局所解析手法を項書き換え系で再現する
- 解法：~~トークン式グラフ書き換え系を項書き換え系で表す~~
- 解法：プログラムを項とみなして項書き換え系を使う
- 課題：項書き換え系には評価戦略がない

```
(1 + 2) × (3 + 4)
-> 3 × (3 + 4)
-> 3 × 7
-> 14
```

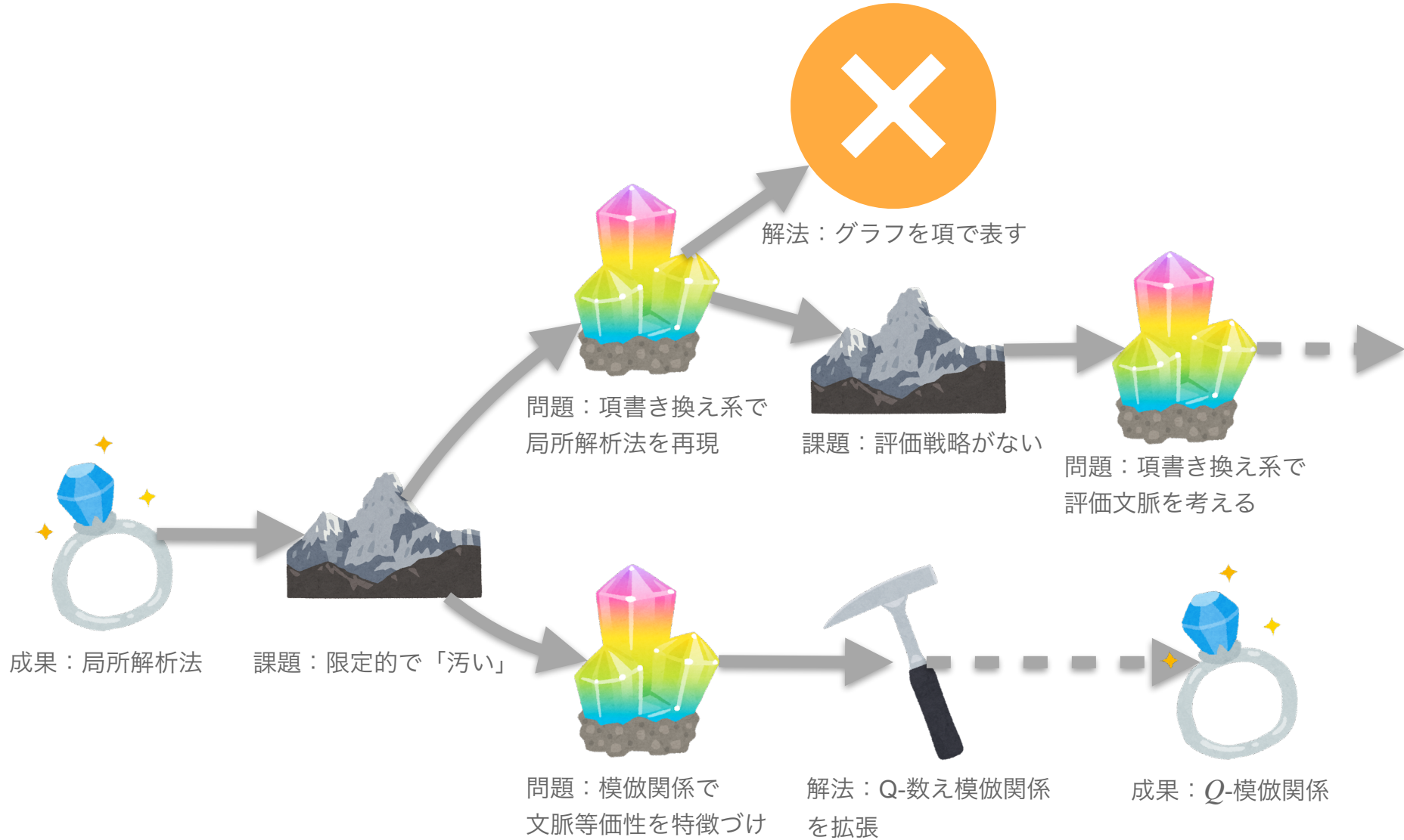
```
(1 + 2) × (3 + 4)
-> (1 + 2) × 7
-> 3 × 7
-> 14
```

新たな課題の発見

- 課題：局所解析手法は限定的で「汚い」
- 問題：局所解析手法を項書き換え系で再現する
- 解法：~~トークン式グラフ書き換え系を項書き換え系で表す~~
- 解法：プログラムを項とみなして項書き換え系を使う
- 課題：項書き換え系には評価戦略がない
- 問題：評価文脈を持つ項書き換え系を作る
- 解法？：「項評価系」
 - 58th TRS Meeting '23で途中経過を発表



学び：
問題を適切に設定できれば
筋書きが見えてくる



学び：
問題を適切に設定できれば
筋書きが見えてくる

駆け出し研究者、作問を学ぶ

- 課題と問題は違う
 - 「（課題）を解消するために、
（問題）を解くような（解法）を発見した」
- 問題を適切に設定できると筋書きが見えてくる
- 「問題を見つけるのが一番大変」なので、人と話そう
- 問題を設定するにもアイデアが要る
- 結局、人と話すのが効いた



Special Thanks

- 上田和紀先生（早稲田大学）
- 浜名誠先生（群馬大学）
- 千葉滋先生（東京大学）
- 蓮尾一郎先生（NII）
- ト部夏木さん（NII）
- 眞田嵩大さん（京都大学）

